

**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ
УНИВЕРСИТЕТ**

«МИФИ»

Кафедра кибернетики

НАУЧНАЯ СТАТЬЯ

**Мультиагентная система на основе техники процедурной мета-
рефлексии (PMR): архитектура, метрики и воспроизводимый протокол
исследования**

Проектная практика, 2-й курс

Авторы:

Коротаев Андрей Дмитриевич

Мандрыка Арина Юрьевна

Черепков Павел Александрович

Паранюшкин Алексей Кириллович

Казанов Амир Ринатович

Москва, 2026

АННОТАЦИЯ

В статье рассматривается проект мультиагентной системы, реализующей технику метакогнитивного промптинга Procedural Meta-Reflection (PMR, процедурная мета-рефлексия).

Цель проекта состоит в проверяемом исследовании того, повышает ли явное разделение процедурного анализа, исполнения и постфактум-рефлексии качество объяснений и воспроизводимость решений по сравнению с одиночным вызовом той же базовой языковой модели.

Система построена как линейный конвейер из трёх специализированных агентов:

- Procedural Analyst формирует план и отвергнутые альтернативы;
- Solver / Executor выполняет задачу по плану;
- Reflective Analyst оценивает согласованность решения с процедурой, выявляет риски и формулирует вердикт.

В работе обобщены исходные методические положения проекта, содержание пояснительной записки, структура программной реализации, состав тестового датасета из 30 задач и воспроизводимый протокол сравнительного эксперимента.

Особое внимание уделено метрикам:

- ROUGE-L;
- экспертным шкалам обоснованности процедурного выбора;
- глубины анализа альтернатив и воспроизводимости;
- стоимости в токенах;
- валидности JSON-выводов;
- времени ответа и межэкспертному согласию по каппе Коэна.

Численные эмпирические результаты в статье сознательно не фабрикуются: документ фиксирует готовый протокол исследования и форму представления результатов, которые должны быть получены при запуске описанной процедуры на зафиксированной версии модели.

***Ключевые слова:** большие языковые модели, метакогнитивный промптинг, Procedural Meta-Reflection, PMR, многоагентная система, Chain-of-Thought, Reflexion, воспроизводимость, ROUGE-L, экспертная оценка.*

1. ВВЕДЕНИЕ

Развитие больших языковых моделей (БЯМ) сделало возможным автоматизированное решение учебных, аналитических и инженерных задач, однако одновременно обострило проблему объяснимости. Модель может дать корректный ответ, но не показать, почему выбрана именно такая процедура решения; может перечислить шаги, но не объяснить, какие альтернативы были рассмотрены; может уверенно сформулировать вывод, не указав границ применимости. Для учебных и исследовательских сценариев это принципиальное ограничение: результат должен быть не только правильным, но и проверяемым, аргументированным, воспроизводимым другим человеком или системой.

Проект «Мультиагентная система на основе техники процедурной мета-рефлексии (PMR)» направлен на исследование этой проблемы в контролируемом масштабе проектной практики. Вместо того чтобы сравнивать разные модели, проект сравнивает две стратегии использования одной и той же БЯМ: **базовый одиночный вызов с минимальной инструкцией** и **трёхэтапный PMR-конвейер**, где модель последовательно выступает в ролях процедурного аналитика, исполнителя и рефлексивного аналитика. Такой дизайн изолирует вклад промптовой и архитектурной организации взаимодействия: если качество объяснений повышается, это нельзя объяснить большей мощностью другой модели, поскольку модель остаётся той же.

Актуальность исследования определяется тремя факторами. Во-первых, метакогнитивные промпты позволяют вынести на поверхность элементы мониторинга, оценки и контроля рассуждения, которые в обычном ответе

остаются неявными. Во-вторых, многоагентное разбиение задачи на роли снижает когнитивную перегрузку одного промпта: один агент проектирует процедуру, второй исполняет, третий оценивает соответствие. В-третьих, образовательный контекст требует воспроизводимости: студент второго курса должен понимать не только итог, но и путь к нему, критерии выбора метода и ограничения предложенного решения.

Цель настоящей статьи — представить целостное научно-техническое описание проекта:

- теоретическую основу PMR;
- постановку задачи;
- архитектуру системы;
- программную реализацию;
- датасет;
- метрики;
- план эксперимента и пошаговую инструкцию воспроизведения.

Статья оформлена как готовый документ для дальнейшего использования: в неё включены рубрикаторы экспертной оценки, формулы агрегирования результатов, контрольные требования к среде запуска и перечень рисков, обнаруженных при анализе кода.

Объектом исследования является мультиагентная система на основе БЯМ, реализующая технику PMR. Предмет исследования — влияние явного процедурного анализа и постфактум-рефлексии на качество ответа, объяснимость, воспроизводимость и стоимость решения. Основная гипотеза формулируется следующим образом: PMR-конвейер должен давать более аргументированные и воспроизводимые ответы, особенно на задачах средней и высокой сложности, но будет требовать большего числа токенов и времени ответа по сравнению с базовой моделью.

2. ТЕОРЕТИЧЕСКАЯ ОСНОВА И МЕСТО PMR СРЕДИ МЕТАКОГНИТИВНЫХ ТЕХНИК

Метакогниция применительно к ИИ понимается как способность системы наблюдать за собственным ходом рассуждения, оценивать качество промежуточных и итоговых выводов и изменять стратегию при необходимости. В классической триаде это соответствует мониторингу, оценке и контролю. Для БЯМ такая способность не является психологическим «самосознанием» модели; в инженерном смысле речь идёт о специальной организации промптов и контекста, которая заставляет модель генерировать явные отчёты о процедуре, основаниях выбора и возможных ошибках.

В исходных методических материалах подчёркивается, что традиционные промпты часто ориентированы только на получение ответа, тогда как метакогнитивный промптинг требует от модели анализа собственных рассуждений и адаптации стратегии. PMR занимает в этой системе особое место: техника направлена не на общую самооценку уверенности и не на бесконечную итеративную коррекцию, а на раскрытие процедурного знания — того, как именно строится решение. Она переводит неявные эвристики модели в явную методику, пригодную для проверки и повторения.

Ключевое различие, на котором основана PMR, — различие декларативного и процедурного знания. Декларативное знание отвечает на вопрос «что известно?», процедурное — на вопрос «как действовать?». В задачах алгоритмики, логики, анализа текста и математики правильный ответ часто зависит не только от факта, но и от корректного выбора процедуры. Например, для поиска кратчайшего пути в графе с неотрицательными весами важно не просто назвать алгоритм Дейкстры, но и объяснить, почему BFS неприменим к взвешенному графу, а алгоритмы для отрицательных весов избыточны.

По отношению к Chain-of-Thought PMR можно рассматривать как расширение: CoT стимулирует пошаговое рассуждение, но не всегда требует

объяснить, почему выбрана именно такая цепочка шагов. По отношению к Reflexion PMR ближе к постфактум-анализу, но принципиально отличается отсутствием замкнутого цикла «решение — критика — новое решение». В рамках данного проекта третий агент не переписывает решение, а формирует аналитический отчёт о согласованности процедуры и исполнения. Это делает эксперимент проще, дешевле и воспроизводимее, но ограничивает способность системы исправлять собственные ошибки.

Для многоагентных систем метакогнитивный слой особенно важен из-за риска каскадных ошибок. Если первый агент ошибочно выбирает процедуру, второй может аккуратно исполнить неверный план, а третий — формально подтвердить согласованность, если его промпт недостаточно критичен. Поэтому PMR-архитектура должна оцениваться не только по красоте объяснения, но и по реальному качеству выбора процедур, глубине анализа альтернатив и способности выявлять риски, а не замазывать их убедительным текстом.

3. ПОСТАНОВКА ЗАДАЧИ И ОБЪЕКТ ИССЛЕДОВАНИЯ

Задача проекта состоит в разработке прототипа мультиагентной системы, реализующей PMR, формировании тестового набора из 30 задач трёх уровней сложности и подготовке сравнительного эксперимента между PMR-системой и базовой БЯМ. Базовая система определяется минимально: одиночный вызов той же модели с инструкцией «Ты — полезный ассистент. Реши задачу: {задача}». PMR-система делает три последовательных вызова той же модели с разными системными ролями и структурированным обменом данными.

Такое сравнение корректно для учебного исследования, потому что контролирует ключевой внешний фактор — качество самой модели. Обе системы используют один вычислительный бэкенд, один набор задач, одну версию модели и фиксированные параметры генерации. Отличается только

способ постановки задачи и организация промежуточных рассуждений. Следовательно, наблюдаемая разница в экспертных метриках может быть интерпретирована как эффект процедурной мета-рефлексии, а не как следствие замены модели.

Система ориентирована на текстовые задачи на русском языке объёмом до 500 символов. Домены датасета: математика, алгоритмы, логика и текстовое рассуждение. Уровни сложности: simple, medium, hard. Простые задачи проверяют базовую корректность и устойчивость формата; средние задачи вводят выбор между несколькими подходами; сложные задачи требуют методологического решения, где особенно важны аргументация процедуры и анализ альтернатив.

Выход PMR-системы имеет три смысловые части. Первая часть — процедурный анализ: тип задачи, план, альтернативы и ограничения. Вторая — выполнение: пошаговое решение с комментариями и итоговым ответом. Третья — рефлексия: оценка согласованности процедуры и исполнения, разбор шагов, риски и финальный вердикт. В научном эксперименте отдельно оценивается не только итоговый ответ, но и качество этих процедурных компонентов.

4. АРХИТЕКТУРА МУЛЬТИАГЕНТНОЙ СИСТЕМЫ

Архитектура системы линейна:



В ней нет циклов, параллельных ветвей и сложного графа управления. Это осознанное проектное решение: для учебного прототипа важнее прозрачность и воспроизводимость, чем функциональная насыщенность.

Каждый агент является отдельным вызовом БЯМ с собственным системным промптом; данные между агентами передаются в JSON-структурах, что уменьшает неоднозначность контекста и упрощает последующую валидацию.



Рисунок 1 — Логика PMR-конвейера

Agent 1, или **Procedural Analyst**, выполняет предварительное проектирование решения. Его задача — не решить задачу, а классифицировать её, предложить план, перечислить альтернативы и объяснить, почему они отклонены. Это соответствует идее пре-рефлексии: модель заранее формирует гипотезу о наиболее подходящей процедуре и фиксирует основания выбора. Такой вывод полезен для последующей проверки: если итоговое решение окажется ошибочным, можно определить, была ли ошибка заложена уже в плане или появилась на этапе исполнения.

Agent 2, или **Solver / Executor**, получает исходную задачу и структурированный результат Agent 1. Он не должен выбирать процедуру с нуля, а обязан следовать заданному плану. Каждый шаг решения описывается через действие и процедурный комментарий. Отдельное поле `adaptation_points` фиксирует случаи, когда план пришлось адаптировать при исполнении. Это поле важно для анализа устойчивости PMR: хороший план не обязательно должен быть жёстким, но его адаптация должна быть явной и объяснённой.

Agent 3, или Reflective Analyst, выполняет постфактум-оценку. Он не решает задачу заново, а проверяет, насколько Agent 2 следовал процедуре Agent 1, какие шаги были выполнены корректно, где возникли проблемы и какие улучшения возможны. В текущей реализации Agent 3 возвращает оценку `alignment_assessment.score` от 0 до 10, массив `step_reflection`, список рисков и `final_verdict`. Эта структура делает рефлекссию пригодной как для чтения человеком, так и для последующего машинного анализа.

Главная архитектурная сила решения — разделение обязанностей. Один длинный промпт мог бы потребовать от модели сразу планировать, решать и оценивать себя, но такая схема хуже контролируется: модель склонна смешивать уровни рассуждения. Трёхагентный конвейер превращает PMR в последовательность проверяемых контрактов. Слабая сторона решения — рост стоимости: три вызова модели потребляют больше токенов, увеличивают задержку и создают риск накопления ошибки от этапа к этапу.

5. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И АНАЛИЗ КОДА

Текущая программная версия реализована на Python и использует OpenAI-совместимый интерфейс Yandex AI Studio. По структуре импортов проект предполагает модульную организацию: корневые файлы *main.py*, *pmr_api.py*, *config.py*, *llm_client.py* и *parser.py*; пакет `agents` с *agent1.py*, *agent2.py*, *agent3.py*; пакет `utils` с *json_parser.py*. При воспроизведении важно сохранить именно такую структуру, иначе импорты вида `agents.agent1` и `utils.json_parser` не будут разрешены.

Файл *config.py* отвечает за конфигурацию.

Он загружает переменные окружения через *python-dotenv*, требует `YANDEX_API_KEY`, `YANDEX_FOLDER_ID` и `YANDEX_MODEL`, задаёт значение `YANDEX_BASE_URL` по умолчанию *https://ai.api.cloud.yandex.net/v1*, а также параметры `TEMPERATURE=0.2` и `MAX_OUTPUT_TOKENS=1500`.

Секреты не зашиваются в код, что является правильной практикой для воспроизводимости и безопасности: исследователь может опубликовать код без раскрытия ключей API.

Файл `llm_client.py` инкапсулирует вызов модели.

Клиент создаётся через OpenAI SDK, а имя модели формируется в формате `gpt://{folder_id}/{model_name}`. Используется endpoint `chat.completions.create`, а не `responses.create`, что соответствует замечанию в коде о правах доступа к OpenAI-совместимому сервису. Функция `_extract_chat_text` обрабатывает основной вариант ответа через `response.choices[0].message.content` и резервный вариант через `response.choices[0].text`.

Файлы `agent1.py`, `agent2.py` и `agent3.py` содержат системные промпты и функции вызова агентов.

Все три агента требуют строгий JSON без текста вне объекта и запрещают Markdown-блоки. Каждый агент имеет до двух попыток: если первый ответ не проходит парсинг или валидацию, формируется повторный запрос с требованием вернуть только валидный JSON. Такой механизм важен для инженерной устойчивости, потому что даже при явной инструкции БЯМ иногда добавляет поясняющий текст вокруг JSON.

Файл `json_parser.py` реализует два уровня обработки: сначала строгий `json.loads`, затем попытку извлечь подстроку между первой открывающей и последней закрывающей фигурной скобкой. После парсинга выполняется схема-валидация.

- Для Agent 1 обязательны `task_type`, `plan`, `alternatives` и `notes`;
- для Agent 2 — `solution_steps`, `final_answer` и `adaptation_points`;
- для Agent 3 — `alignment_assessment`, `step_reflection`, `risks` и `final_verdict`.

Это повышает воспроизводимость, потому что ошибки формата фиксируются явно, а не маскируются в свободном тексте.

Файл `pmr_api.py` предоставляет программный интерфейс: `run_agent1`, `run_agent2`, `run_agent3`, `run_pipeline` и `run_pipeline_v3`.

Полный PMR-конвейер соответствует `run_pipeline_v3`.

Файл `main.py` реализует CLI-режимы:

- одиночный запуск Agent 1;
- запуск Agent 1 → Agent 2 через `--agent2`;
- запуск полного Agent 1 → Agent 2 → Agent 3 через `--agent3` или `--agent2 --with-agent3`;
- запуск по датасету через `--dataset`.

При анализе кода выявлен важный момент для воспроизводимости эксперимента: `dataset-mode` в `main.py` сейчас вызывает `run_pipeline(task_text)`, то есть Agent 1 → Agent 2, и извлекает `ai_final_answer` из `agent2_result`. Полный Agent 3 при массовом запуске датасета не используется. Для финального исследования PMR в трёхагентной постановке необходимо либо изменить `dataset-mode` на `run_pipeline_v3`, либо явно указать, что массовый датасетный режим измеряет только двухагентную часть системы. В настоящей статье основной протокол предполагает именно полный вариант Agent 1 → Agent 2 → Agent 3, поскольку он соответствует заявленной архитектуре.

Компонент	Назначение	Ключевые данные / выход
<code>config.py</code>	Загрузка переменных окружения и параметров генерации	YANDEX_API_KEY, YANDEX_FOLDER_ID, YANDEX_MODEL, TEMPERATURE, MAX_OUTPUT_TOKENS

llm_client.py	Универсальный вызов OpenAI-compatible API	Текст ответа модели из chat.completions.create
agent1.py	Предварительный процедурный анализ	task_type, plan, alternatives, notes
agent2.py	Исполнение задачи по плану	solution_steps, final_answer, adaptation_points
agent3.py	Постфактум-рефлексия и оценка согласованности	alignment_assessment, step_reflection, risks, final_verdict
json_parser.py	Парсинг и валидация JSON-ответов	Валидированные словари Python или явная ошибка
pmr_api.py	Импортируемый API конвейера	run_pipeline, run_pipeline_v3
main.py	CLI-запуск и режим работы с датасетом	--agent2, --agent3, --dataset --all, --dataset --nums

Таблица 1 — Основные программные компоненты

6. ДАТАСЕТ ИССЛЕДОВАНИЯ

Тестовый датасет содержит 30 задач: 10 простых, 10 средних и 10 сложных. Распределение по доменам: algorithms — 10, logic — 7, text_reasoning — 7, math — 6.

Такая структура позволяет проверить систему не только на арифметических задачах, где фактическая правильность легко оценивается автоматически, но и на задачах выбора процедуры, где важна экспертная оценка рассуждения.

Простые задачи выполняют роль sanity-check: модель должна безошибочно решать элементарные вопросы и сохранять требуемый формат. Средние задачи проверяют умение выбрать между очевидными альтернативами, например сортировкой и подсчётом частот или множеством и сортировкой для поиска повторов. Сложные задачи проверяют методологическое мышление: выбор алгоритма для графа, различение жадного и динамического подхода, оценку воспроизводимости инструкции, выбор процедуры анализа зависимости.

Важно, что поле answer из датасета не должно передаваться в модель во время генерации ответа. Оно используется только после завершения решения как эталон для автоматического и экспертного сравнения. Это требование отражено в README проекта и должно быть сохранено в экспериментальном runner-скрипте. Нарушение этого правила приведёт к утечке эталона и сделает результаты недействительными.

7. ОПИСАНИЕ И АРГУМЕНТАЦИЯ МЕТРИК

Метрики исследования должны покрывать две стороны качества: фактическую близость ответа к эталону и процедурную ценность PMR. Если измерять только правильность итогового ответа, преимущество PMR может оказаться невидимым: базовая модель часто справляется с простыми задачами без дополнительной рефлексии. Если измерять только структурированность, PMR получит искусственное преимущество, потому что структура встроена в промпты. Поэтому набор метрик разделяется на автоматические, экспертные и инженерные.

Группа	Метрика	Что измеряет	Аргументация
Автоматические	ROUGE-L	Лексическое совпадение ответа с эталоном по наибольшей общей подпоследовательности	Даёт воспроизводимую числовую оценку близости к эталону, особенно полезную для

			коротких ответов и формулировок
Автоматические	Semantic / expert correctness	Содержательная правильность итогового ответа	Компенсирует ограничение ROUGE-L: правильный ответ может быть переформулирован иначе
Экспертные	Обоснованность процедурного выбора, 1–5	Насколько убедительно выбран метод решения с учётом условий задачи	Центральная PMR-метрика: техника должна улучшать не только ответ, но и выбор процедуры
Экспертные	Глубина анализа альтернатив, 1–5	Полноту и корректность рассмотрения отвергнутых подходов	Проверяет, действительно ли PMR раскрывает пространство решений, а не просто пишет план
Экспертные	Воспроизводимость, 1–5	Можно ли по описанию независимо повторить решение	Ключевая метрика для учебного и научного применения
Инженерные	Total tokens	Суммарный расход входных и выходных токенов	Нужен для честного сравнения с базовой моделью, так как PMR делает три вызова
Инженерные	JSON validity rate	Доля ответов, прошедших парсинг и схему валидации	Измеряет надёжность системы как программного компонента
Инженерные	Retry count / failure rate	Частоту повторных вызовов и отказов	Показывает устойчивость промптов и API-обвязки
Инженерные	Latency	Время ответа на задачу	Позволяет оценить практическую применимость PMR в интерактивных сценариях

Таблица 2 — Набор метрик исследования

ROUGE-L выбран как базовая автоматическая метрика, потому что он прост, воспроизводим и не требует дополнительной модели-оценщика. Однако его нельзя считать достаточным: он измеряет текстовое пересечение, а не логическую правильность. Поэтому ROUGE-L используется как вспомогательный индикатор, а не как единственный критерий успеха. Для задач с несколькими эталонами берётся максимум по эталонам: $ROUGE-L(answer, reference) = \max_i ROUGE-L(answer, reference_i)$.

Экспертные метрики вводятся потому, что предмет исследования — не только факт ответа, но и качество процедуры. Обоснованность процедурного выбора оценивает, объясняет ли система, почему выбран именно этот метод. Глубина анализа альтернатив оценивает, различает ли система допустимые, избыточные и ошибочные подходы. Воспроизводимость оценивает, можно ли по ответу повторить решение без дополнительных уточнений. Все три метрики оцениваются по шкале 1–5 двумя независимыми экспертами.

Для исключения субъективности необходим рубрикатор. Балл 1 означает отсутствие критерия или грубую ошибку; балл 2 — поверхностное упоминание без связи с задачей; балл 3 — приемлемое, но неполное объяснение; балл 4 — хорошее объяснение с незначительными пробелами; балл 5 — полное, точное и воспроизводимое обоснование. Если каппа Коэна между экспертами ниже 0,6, рубрикатор пересматривается, спорные случаи обсуждаются и оценивание повторяется.

Балл	Обоснованность выбора	Глубина альтернатив	Воспроизводимость
1	Метод выбран без объяснения или выбран неверно	Альтернативы отсутствуют или ошибочны	Решение невозможно повторить по тексту
2	Есть общая фраза, но связь с условиями слабая	Упомянута одна альтернатива без анализа	Повторение возможно только с догадками
3			

	Выбор в целом понятен, но часть условий не учтена	Альтернативы названы, но отклонение неполное	Основные шаги есть, но нужны уточнения
4	Выбор хорошо аргументирован, есть малые пробелы	Рассмотрены релевантные альтернативы	Процедура в основном воспроизводима
5	Метод выбран строго по условиям, границы применимости ясны	Альтернативы полны, корректно сравнены и отклонены	Другой исполнитель может повторить решение без дополнительных пояснений

Таблица 3 — Рубрикатор экспертных метрик

Метрика стоимости в токенах принципиальна для честной интерпретации. PMR почти неизбежно увеличивает длину промптов и ответов. Поэтому улучшение экспертных метрик должно рассматриваться вместе с коэффициентом «качество / стоимость». Например, можно рассчитывать относительный выигрыш: $\Delta Q = Q_{\text{PMR}} - Q_{\text{base}}$ и нормировать его на отношение токенов: $\text{Efficiency} = \Delta Q / (\text{Tokens}_{\text{PMR}} / \text{Tokens}_{\text{base}})$. Такая метрика не заменяет основную оценку, но показывает, оправдан ли рост затрат.

Структурированность ответа намеренно не должна быть основной метрикой сравнения, поскольку PMR по конструкции всегда генерирует три компонента, а базовая модель не обязана это делать. Включение такой метрики создало бы артефактное преимущество PMR. Вместо этого структурированность проверяется как инженерное условие валидности: если PMR не вернула корректный JSON или пропустила обязательные поля, запуск считается технически проблемным.

8. ЭКСПЕРИМЕНТАЛЬНЫЙ ДИЗАЙН И СТАТИСТИЧЕСКАЯ ОБРАБОТКА

Эксперимент проводится на 30 задачах. Для каждой задачи выполняются два режима: baseline и PMR. Baseline — одиночный вызов модели с минимальной инструкцией. PMR — полный конвейер `run_pipeline_v3`. Для каждого запуска фиксируются исходная задача, версия модели, параметры генерации, полный JSON-вывод всех агентов, итоговый ответ, эталон, число входных и выходных токенов, время ответа, количество повторных попыток и статус парсинга.

Порядок задач должен быть зафиксирован заранее. Чтобы уменьшить влияние временных колебаний API, рекомендуется выполнять baseline и PMR в одном экспериментальном окне и сохранять timestamp каждого запроса. Если API поддерживает seed, он должен быть зафиксирован. Если seed не поддерживается, воспроизводимость обеспечивается через фиксацию версии модели, параметров температуры, полного набора промптов и исходных данных.

Для экспертных метрик используется непараметрический U-критерий Манна–Уитни или парный непараметрический тест, если оценки сопоставляются по одной и той же задаче. Для ROUGE-L и токенов рассчитываются среднее, медиана, стандартное отклонение и межквартильный размах. Для инженерных отказов рассчитывается доля успешных запусков, доля валидных JSON-ответов и среднее число retry на задачу. Отдельно строится таблица ошибок: неверный план Agent 1, неверное исполнение Agent 2, слабая рефлексия Agent 3, технический сбой парсинга.

Согласованность экспертов проверяется каппой Коэна по каждой экспертной метрике. Значение $\kappa < 0,6$ означает недостаточную согласованность рубрикатора; $0,6 \leq \kappa < 0,8$ — приемлемую; $\kappa \geq 0,8$ — высокую. При низкой согласованности нельзя делать сильные выводы о преимуществе

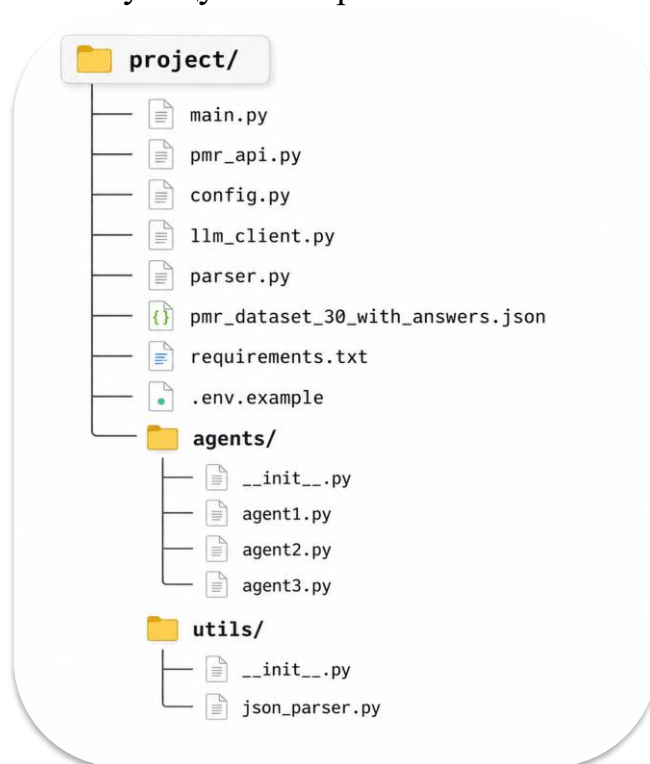
PMR, даже если средние баллы кажутся выше. Это особенно важно для метрик глубины альтернатив и воспроизводимости, где субъективность выше, чем при оценке фактической правильности.

9. ИНСТРУКЦИИ ПО ВОСПРОИЗВЕДЕНИЮ ИССЛЕДОВАНИЯ

Воспроизводимость исследования является центральным требованием. Ниже приведён протокол, позволяющий другому участнику повторить эксперимент при наличии доступа к тому же API или совместимому провайдеру. Все действия должны выполняться в чистой среде, с сохранением логов и без изменения тестового датасета после начала эксперимента.

Шаг 1. Подготовить структуру проекта.

Если файлы были получены из архива, необходимо сохранить пакетную организацию, соответствующую импортам:



Шаг 2. Создать виртуальное окружение и установить зависимости:

```
terminal - project setup

$ python -m venv .env
# Windows:
> .env\Scripts\activate
# Linux/macOS:
$ source .env/bin/activate
$ python -m pip install -r requirements.txt
```

Шаг 3. Настроить переменные окружения.

Нужно скопировать `.env.example` в `.env` и заполнить обязательные значения. Секретные ключи не должны попадать в репозиторий или итоговую статью.

```
.env - project configuration

YANDEX_API_KEY=...
YANDEX_FOLDER_ID=...
YANDEX_MODEL=yandexgpt-lite
YANDEX_BASE_URL=https://ai.api.cloud.yandex.net/v1
TEMPERATURE=0.2
MAX_OUTPUT_TOKENS=1500
```

Шаг 4. Проверить одиночные режимы запуска. Для Agent 1:

```
terminal - запуск Agent 1

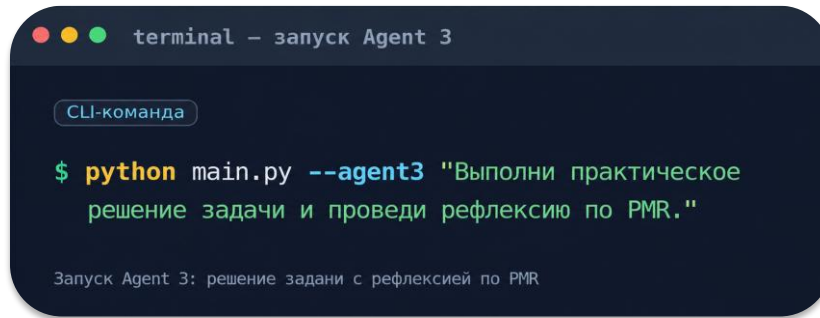
CLI-команда
$ python main.py "Сформируй процедуру решения задачи по классификации текста на 3 категории."
Запуск первого агента: формирование процедуры решения
```

Для Agent 1 → Agent 2:

```
terminal - запуск Agent 1 → Agent 2

CLI-команда
$ python main.py --agent2 "Выполни практическое решение задачи: предложи шаги и действия для классификации текстов на 3 категории."
Запуск связки Agent 1 → Agent 2
```

Для полного PMR-конвейера Agent 1 → Agent 2 → Agent 3:



```
terminal - запуск Agent 3

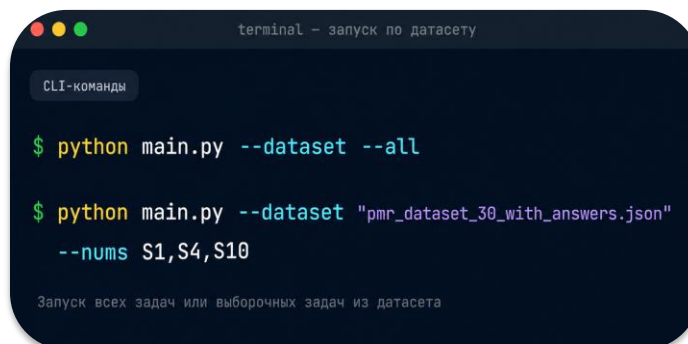
CLI-команда

$ python main.py --agent3 "Выполни практическое
  решение задачи и проведи рефлекссию по PMR."

Запуск Agent 3: решение задани с рефлексией по PMR
```

Шаг 5. Запустить датасет.

Текущий README описывает массовый запуск так:



```
terminal - запуск по датасету

CLI-команды

$ python main.py --dataset --all

$ python main.py --dataset "pmr_dataset_30_with_answers.json"
  --nums S1,S4,S10

Запуск всех задач или выборочных задач из датасета
```

Однако для полного трёхагентного эксперимента необходимо убедиться, что массовый режим вызывает `run_pipeline_v3`, а не `run_pipeline`. Рекомендуемое изменение: в `_run_dataset_mode` заменить `pipeline_result = run_pipeline(task_text)` на `pipeline_result = run_pipeline_v3(task_text)`, сохранять `agent1_result`, `agent2_result` и `agent3_result`, а итоговый ответ брать из `agent2_result.final_answer`, поскольку Agent 3 является оценщиком, а не решателем.

Шаг 6. Сохранить результаты в машинно-читаемом формате.

Минимальный JSONL-лог одной строки должен содержать:

- Id;
- difficulty;
- domain;
- task;
- system_type;

- model;
- temperature;
- max_tokens;
- raw_response;
- parsed_response;
- final_answer;
- reference_answer;
- prompt_tokens;
- completion_tokens;
- total_tokens;
- latency_sec;
- retry_count;
- parse_status;
- timestamp.

Для baseline system_type=baseline, для PMR system_type=pmr.

Шаг 7. Выполнить автоматическую оценку.

Для ROUGE-L использовать один и тот же скрипт и одну схему нормализации текста: привести к нижнему регистру, удалить лишние пробелы, но не удалять значимые математические символы. Для задач с несколькими эталонами брать максимум. Если в текущем датасете по одной ссылке на задачу, поле reference_answer равно answer.

Шаг 8. Выполнить экспертную оценку.

Экспертам выдаются обезличенные ответы без указания, какой системой они получены. Каждый эксперт независимо выставляет баллы по трём PMR-метрикам и, при необходимости, по содержательной правильности. После этого рассчитываются средние значения, медианы и каппа Коэна. Только

после завершения оценивания раскрывается принадлежность ответов к baseline или PMR.

10. ОГРАНИЧЕНИЯ, РИСКИ И НАПРАВЛЕНИЯ РАЗВИТИЯ

Первое ограничение — отсутствие настоящей внутренней метакогниции. PMR управляет текстовым поведением модели через промпты, но не имеет прямого доступа к внутренним вероятностным сигналам, активациям, loss или градиентам. Поэтому рефлексия Agent 3 является сгенерированным аналитическим текстом, а не гарантированной диагностикой истинных причин ошибки. Это не обесценивает технику, но требует аккуратной интерпретации результатов.

Второе ограничение — линейность архитектуры. Конвейер не возвращает решение на доработку после рефлексии. Если Agent 3 обнаружил проблему, система лишь сообщает о ней, но не исправляет ответ. Это упрощает эксперимент и снижает риск неконтролируемых итераций, но ограничивает практическую полезность в задачах, где требуется улучшение итогового ответа.

Третье ограничение — малый размер датасета. 30 задач достаточно для учебного проекта и первичной проверки гипотезы, но недостаточно для сильных обобщений о БЯМ в целом. Результаты нужно рассматривать как пилотное исследование. Для развития проекта датасет следует расширить, сбалансировать по доменам, добавить несколько независимых эталонов и включить задачи, где базовая модель часто ошибается.

Четвёртый риск — рассогласование документации и реализации. Пояснительная записка описывает планируемые параметры отдельных агентов, LangChain, Streamlit, pandas и scipy, тогда как текущий requirements.txt содержит только openai и python-dotenv, а код реализует минимальный CLI/API-прототип. Это нормально для стадии прототипа, но в статье и при

защите нужно ясно различать реализованное состояние и планируемое расширение.

Перспективы развития включают: добавление режима `run_pipeline_v3` для массового датасетного запуска; логирование токенов и времени; отдельный `evaluation.py` для расчёта метрик; веб-интерфейс Streamlit; расширенный датасет; поддержку нескольких эталонов; `blinded expert review`; добавление опционального четвёртого агента-редактора, который исправляет решение только после обнаруженной ошибки Agent 3.

11. ЗАКЛЮЧЕНИЕ

Проект PMR демонстрирует инженерно понятный способ повысить объяснимость работы БЯМ без дообучения модели. Система не пытается сделать модель «сознательной», а дисциплинирует её вывод: сначала явный план, затем решение по плану, затем рефлексия по согласованности. Для образовательных задач это особенно ценно, потому что студент видит не только ответ, но и процедуру, альтернативы, ограничения и риски.

Главный научный вклад проекта заключается в воспроизводимой постановке сравнительного эксперимента. Сравнивается одна и та же модель в двух режимах, используется фиксированный датасет, вводятся автоматические, экспертные и инженерные метрики, а также предусматривается проверка межэкспертного согласия. Такой подход превращает промпт-инженерию из набора удачных формулировок в проверяемую экспериментальную практику.

Главный инженерный результат — работающий прототип CLI/API с тремя агентами, строгими JSON-контрактами и повторными попытками при ошибках формата. При этом анализ кода показывает, какие доработки нужны для финального исследования: полный датасетный запуск через `run_pipeline_v3`, сохранение метаданных, подсчёт токенов, автоматический расчёт ROUGE-L и отдельный модуль статистики. После этих доработок

проект может быть использован как полноценная учебная лаборатория по метакогнитивному промптингу.

Итоговая рекомендация: для защиты и дальнейшей публикации следует позиционировать работу как пилотное исследование процедурной мета-рефлексии в многоагентной БЯМ-системе, не заявляя неподтверждённых численных преимуществ до запуска эксперимента. Сильная сторона проекта — не обещание универсального улучшения, а прозрачный, воспроизводимый и честно измеримый протокол проверки PMR.

СПИСОК ИСТОЧНИКОВ

- [1] Пояснительная записка к эскизному проекту программы «Мультиагентная система на основе техники процедурной мета-рефлексии (PMR)». — НИЯУ МИФИ, кафедра кибернетики, 2026.
- [2] Душкин Р. В. Метакогнитивная промпт-инженерия: практическое руководство по техникам взаимодействия с большими языковыми моделями. — 2025.
- [3] Душкин Р. В. Метакогниция в больших языковых моделях, агентах и многоагентных системах: обзор для курса «Проектная практика в области метакогнитивной промпт-инженерии». — 2026.
- [4] README.md проекта PMR (Procedural Meta-Reflection) — Agent 1 + Agent 2 + Agent 3.
- [5] Исходный код проекта PMR: config.py, llm_client.py, main.py, pmr_api.py, parser.py, utils/json_parser.py, agents/agent1.py, agents/agent2.py, agents/agent3.py.
- [6] Тестовый датасет pmr_dataset_30_with_answers.json. — 30 задач с эталонными ответами.
- [7] ГОСТ 19.404–79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. — М.: Издательство стандартов, 1979.
- [8] ГОСТ 19.105–78. Единая система программной документации. Общие требования к программным документам. — М.: Издательство стандартов, 1978.
- [9] Wei J., Wang X., Schuurmans D. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models // Advances in Neural Information Processing Systems. — 2022. — Vol. 35. — P. 24824–24837.
- [10] Shinn N., Cassano F., Gopinath A. et al. Reflexion: Language Agents with Verbal Reinforcement Learning // Advances in Neural Information Processing Systems. — 2023. — Vol. 36.
- [11] Lin C.-Y. ROUGE: A Package for Automatic Evaluation of Summaries // Proceedings of ACL Workshop on Text Summarization Branches Out. — 2004. — P. 74–81.
- [12] Wang X., Wei J., Schuurmans D. et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models // International Conference on Learning Representations. — 2023.